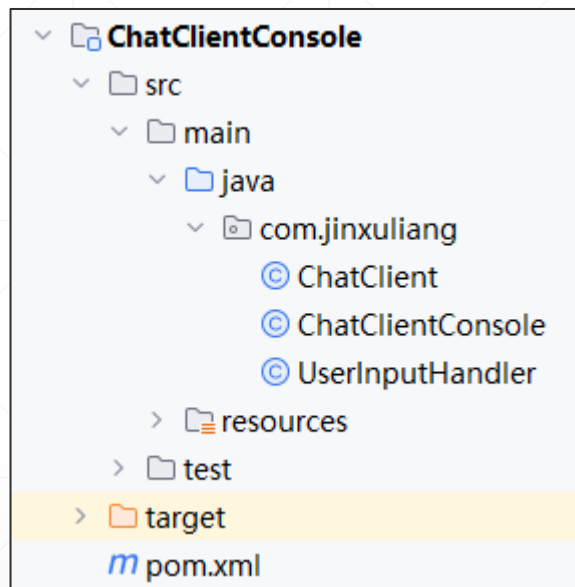


开发控制台聊天客户端

北京理工大学计算机学院
金旭亮

项目结构



ChatClientConsole: 程序入口点

UserInputHandler: 接收用户输入,
发送信息

ChatClient: 客户端聊天主控流程

程序入口点

```
public class ChatClientConsole {  
  
    public static void main(String[] args) throws IOException {  
        // 等待用户输入消息  
        BufferedReader consoleReader =  
            new BufferedReader(new InputStreamReader(System.in));  
        System.out.print("请输入您的名字(直接回车采用系统生成的名字) :");  
        String userName = consoleReader.readLine();  
        if (userName.isEmpty()) {  
            userName = UUID.randomUUID().toString();  
        }  
        System.out.print("请输入服务器的IP地址:");  
        String serverIP = consoleReader.readLine();  
        System.out.println("本用户名:" + userName);  
        ChatClient chatClient = new ChatClient(serverIP, userName);  
        chatClient.start();  
    }  
}
```

ChatClient

```
public class ChatClient {  
    7 usages  
    private Socket socket;  
    //接收消息  
    2 usages  
    private BufferedReader reader;  
    //发送消息  
    5 usages  
    private BufferedWriter writer;  
  
    2 usages  
    private String serverIP;  
    2 usages  
    private String userName;  
  
    1 usage  
    public ChatClient(String serverIP, String userName) {  
        this.serverIP = serverIP;  
        this.userName = userName;  
    }  
}
```

```
// 检查用户是否准备退出  
1 usage  
public boolean readyToQuit(String msg) {  
    return Constants.QUIT.equals(msg);  
}  
  
// 关闭Socket并退出  
1 usage  
public void close() {  
    if (writer != null && !socket.isClosed()) {  
        try {  
            System.out.println("关闭socket");  
            writer.close();  
        } catch (IOException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

ChatClient-发送消息

// 发送消息给服务器

1 usage

```
public void send(String msg) throws IOException {  
    ChatMessage messageToBeSent = null;  
    if (!socket.isOutputShutdown()) {  
        messageToBeSent = ChatMessage.builder().userName(userName)  
            .message(msg)  
            .build();  
        writer.write(str: ChatMessage.toJson(messageToBeSent) + "\n");  
        writer.flush();  
    }  
}
```

ChatClient-接收消息

```
public String receive() {  
    String msg = null;  
    if (!socket.isInputShutdown() && !socket.isClosed()) {  
        try {  
            msg = reader.readLine();  
        } catch (IOException e) {  
            }  
    }  
    return msg;  
}
```

ChatClient-主控流程

```
public void start() {
    try {
        // 创建socket
        socket = new Socket(serverIP, Constants.DEFAULT_SERVER_PORT);
        // 创建IO流
        reader = new BufferedReader(
            new InputStreamReader(socket.getInputStream())
        );
        writer = new BufferedWriter(
            new OutputStreamWriter(socket.getOutputStream())
        );
        // 处理用户的输入
        new Thread(new UserInputHandler(chatClient: this)).start();
        // 读取服务器转发的消息
        String msg = null;
        while ((msg = receive()) != null) {
            ChatMessage message = ChatMessage.fromJson(msg);
            System.out.println("\n" + message.getUserName() + ":" + message.getMessage());
            System.out.print(">");
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```